Audio Recognition on ESC-50

Team 6

		
Sirui Sun	Eugenio	Rachid
• Data Preprocessing • Baseline • CNN	• Experimental comparison • Evaluation	• Existing Encoder • Fine-tuning • Feature extraction

Highlights (4–6):

◎**Explored two distinct paradigms for audio classification:**
- A convolutional neural network (CNN) trained from scratch.
- A transfer learning approach using **OpenL3 pretrained embeddings.

◎**Demonstrated the value of pretrained audio representations(OpenL3):**
Outperformed from-scratch CNN in both accuracy and training efficiency on the ESC-50 dataset.

◎**Implemented a full audio classification pipeline, including:**
- Feature extraction (log-mel, MFCC, ZCR, delta)
- Data preprocessing
- Model evaluation with baselines (LogReg / Random Forest)

◎**Gained hands-on experience with audio machine learning, including:**
- Handling real-world environmental sound data
- Visualizing and interpreting mel spectrograms
- Applying data augmentation and regularization to combat overfitting

◎**Developed a modular and extensible codebase, organized into clear components: data_loading, baseline models, CNN architectures, and transfer learning.**

Brief Report:

Keywords: **CNN**, **OpenL3**, **transfer learning**, **audio classification**,**fine-tuning, log-Mel spectrogram**, **data augmentation, Segmentationdata , ESC-50 dataset,**

This project focuses on performing audio classification on the ESC-50 dataset. Our primary goal is to apply CNN techniques learned in class and pretrained model , particularly in the context of audio data, which is generally less intuitive than image data. Rather than aiming to surpass the state-of-the-art performance—since recent methods have already achieved over 98% accuracy on the test set using pretrained models—we emphasize practical understanding and model comparison.

Our work primarily involves implementing data segmentation and augmentation from scratch to construct a preprocessed dataset, and building a modular CNN-based audio classifier. While our CNN architecture is broadly inspired by Piczak's 2015 design, we made several modifications based on empirical observations. Since no official implementation was provided in the original paper, we implemented the entire pipeline ourselves, including data handling, CNN model construction, training, and visualization.

Compared to the baseline models (logistic regression: 6%, random forest: 21% on the test set), our from-scratch CNN achieved 60.5% test accuracy and nearly 99% training accuracy. A key insight we gained is the importance of frequency structure and temporal locality in audio classification. Since CNNs operate along both frequency and time axes, we found it beneficial to use asymmetric convolution kernels—wider in frequency, narrower in time—to better capture global frequency patterns and local temporal dynamics.

Besides, the gap between test accuracy and training accuracy reveals a clear overfitting issue, despite our efforts to mitigate it through aggressive segmentation (increasing training samples from 2,000 to over 14,000) and using probability voting for final label assignment per audio clip.We suspect this is due to the limited audio duration and small dataset size, which causes some audio segments to be overly short and contextually ambiguous. Additionally, our model remains relatively shallow, consisting of only two convolutional layers and one fully connected layer.

In addition to our from-scratch CNN, we explored a transfer learning approach using the OpenL3 pretrained audio encoder. OpenL3 was originally trained to learn audio-visual representations from unlabeled YouTube videos, making it highly effective at extracting semantically rich features from sound. Rather than training an entire model from raw audio, we used OpenL3 to extract 512-dimensional embeddings from each audio clip in ESC-50. These embeddings capture high-level characteristics of the sound, such as tone, texture, and structure, making them suitable inputs for a lightweight classifier.

We first evaluated the effectiveness of these embeddings by training a logistic regression model on them. This configuration already achieved significantly higher accuracy than traditional benchmarks, demonstrating the value of pre-trained features. Next, to determine whether adapting the encoder to our specific task could improve performance, we implemented several fine-tuning strategies: unfreezing the last 10 layers or the last 15 layers of the OpenL3 model. We trained these hybrid models on ESC-50 using small, dense layers stacked on top of the encoder.

We observed that partial fine-tuning produced modest but consistent improvements in test accuracy. These experiments confirmed the robustness of the pre-trained models in low-resource scenarios. Compared to our original CNN, the OpenL3-based approach offered greater robustness and superior performance with significantly fewer parameters to train.

Main references beyond course materials:

- Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. https://github.com/karolpiczak/ESC-50
- Cramer et al. (2019). Look, Listen, and Learn More: A multimodal L3 embedding for audio-visual recognition.
- OpenL3 GitHub repository: https://github.com/marl/openl3

Description of Appendices (Optional):

All data and code required for reproducing our full pipeline have been uploaded to a public GitHub repository created by our team. You are welcome to browse the repository. (https://github.com/Sirui97/ML-Project)

The README provides a detailed explanation of the repository structure.
Key files to refer to include:

1. Data_Loader/loader.py – data preprocessing and feature extraction
2. Model/baseline.py – baseline models (logistic regression and random forest)
3. Model/CNN.py – CNN model trained from scratch
4. Phase3/ – experiments with pretrained models and fine-tuning

Audio Recognition on ESC-50

Group6: Sirui,Eugenio,Rachid



Code and data are publicly available at our GitHub repository:
https://github.com/Sirui97/ML-Project

A glimpse of ‘ESC-50’



- The ESC-50 dataset is a **labeled** collection of **2000 environmental audio recordings** suitable for benchmarking methods of environmental sound classification.

- The dataset consists of **5-second-long recordings** organized **into 50 semantical classes** (with 40 examples per class) loosely arranged into 5 major categories

- Source：Karol J. Piczak,2015
- Paper：ESC: Dataset for Environmental Sound Classification
- github:https://github.com/karolpiczak/ESC-50

Animals	Natural soundscapes & water sounds	Human, non-speech sounds	Interior/domestic sounds	Exterior/urban noises
Dog	Rain	Crying baby	Door knock	Helicopter
Rooster	Sea waves	Sneezing	Mouse click	Chainsaw
Pig	Crackling fire	Clapping	Keyboard typing	Siren
Cow	Crickets	Breathing	Door, wood creaks	Car horn
Frog	Chirping birds	Coughing	Can opening	Train
Cat	Water drops	Footsteps	Washing machine	
Hen	Wind	Laughing	Vacuum cleaner	Church bells
Insects (flying)	Pouring water	Brushing teeth	Clock alarm	Airplane
Sheep	Toilet flush	Snoring	Clock tick	Fireworks
Crow	Thunderstorm	Drinking, sipping	Glass breaking	Hand saw

- 50categories*40 samples=2000 samples in total

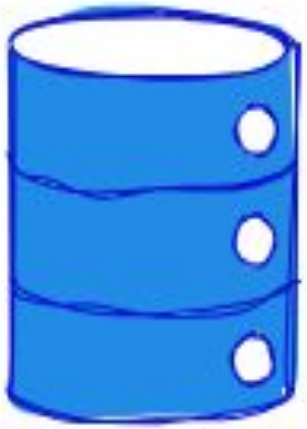
Goal:
perform environmental sound classification across the 50 categories in the ESC-50 dataset.



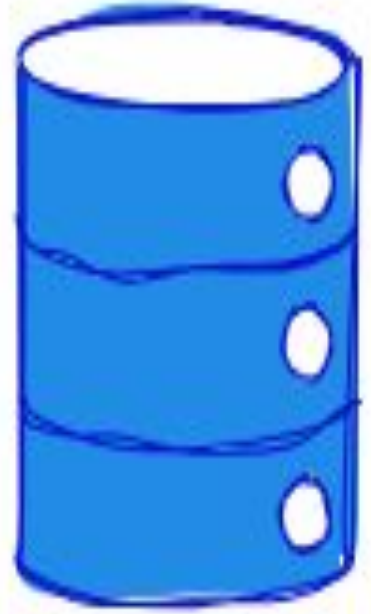
Baseline (Logistic,RandomForest)



CNN from scratch



pretrained model and finetuning



in terms of performance

Data-Preprocessing

- 1) 5-second clips with varying silence — **many useless parts!!**
- 2) Only 2,000 samples for 50 classes — **data is limited!!**
- 3) Audio → features: **less intuitive than images**

Solutions:
Build 2 classes:

```
class AudioFeatureExtractor:
```

```
class ESC50Dataset(Dataset)
```

class AudioFeatureExtractor:

- Constructs 4-channel features
- audio augmentation:
  - Time Shift: simulate slight temporal offset
  - Pitch Shift: adjust pitch ±2 semitones without changing speed
- Normalization

class ESC50Dataset(Dataset)

- Remove silence from raw audio
- Split into overlapping segments
- Preprocess before training → saves time during training



Now we have 14,329 uniform-length segments!
No more useless silence!

Features outline:

- Log Mel Spectrogram

  Human-hearing-inspired frequency representation

- Delta of Log Mel

    Temporal derivative → captures dynamic changes

- MFCC (Mel-Frequency Cepstral Coefficients)

  DCT of log mel → captures timbre (spectral envelope)

- Zero Crossing Rate

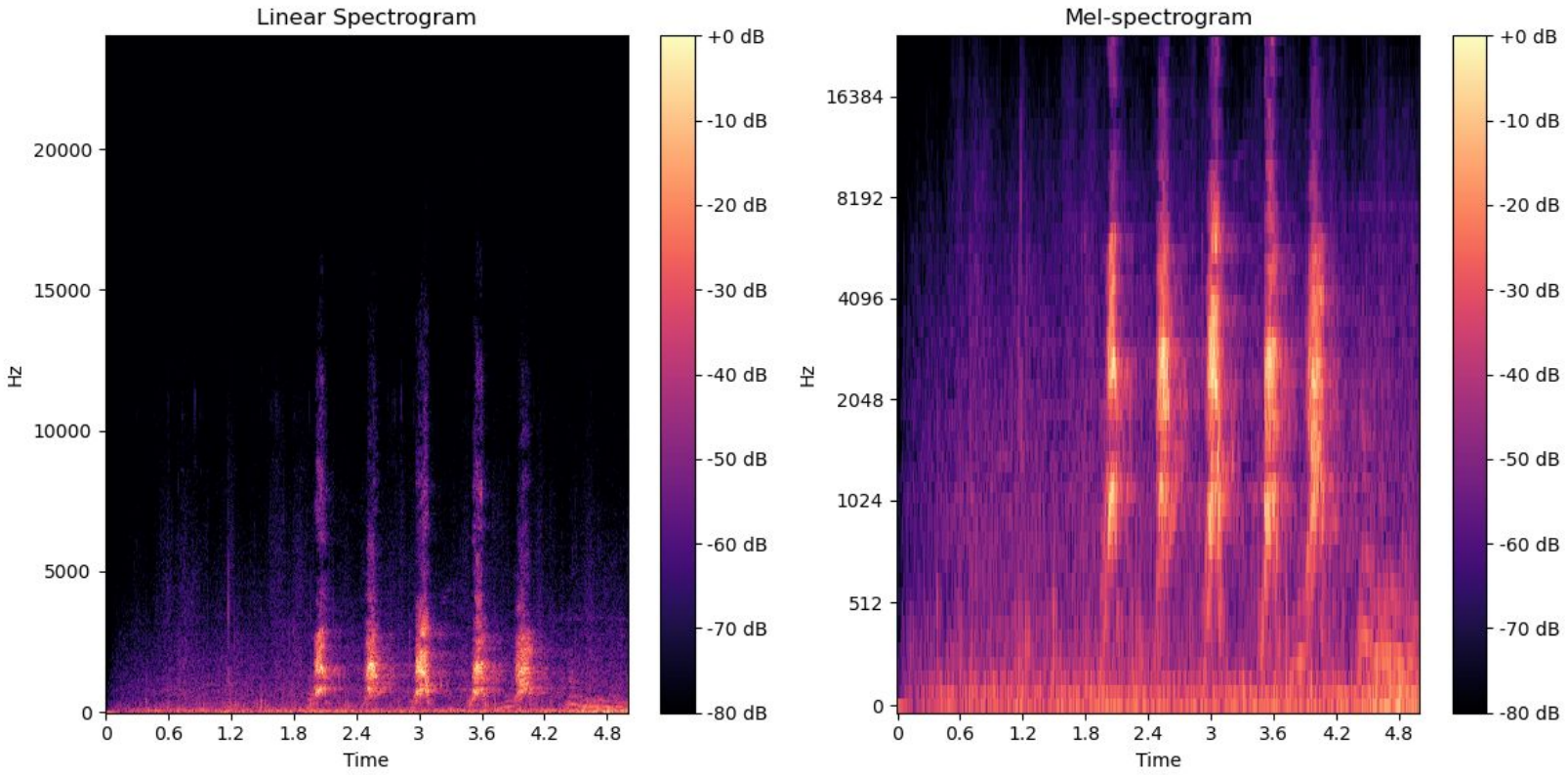
  Counts how often the signal crosses zero

  Reflects roughness / percussiveness

Features outline:

- Log Mel Spectrogram

  Human-hearing-inspired frequency representation



example of an audio clip in ESC-50(Dog)

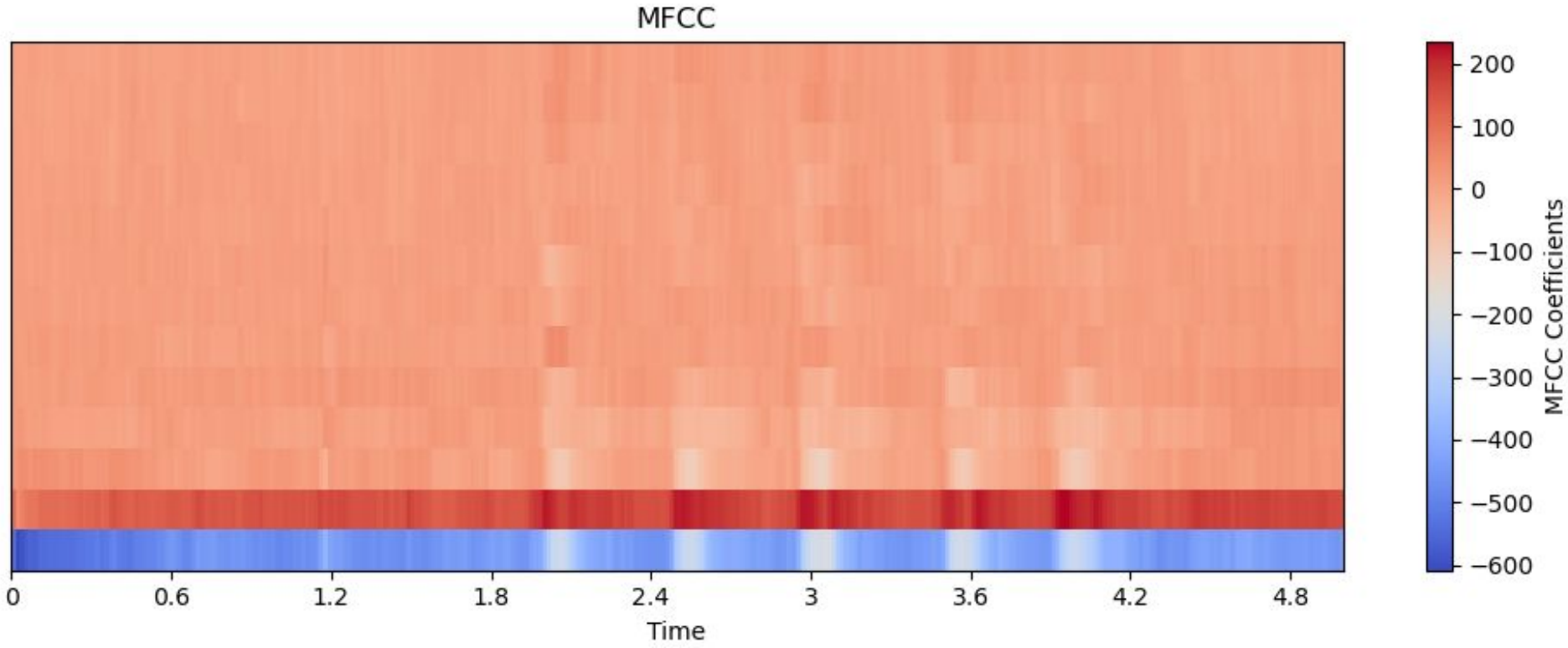
Features outline:

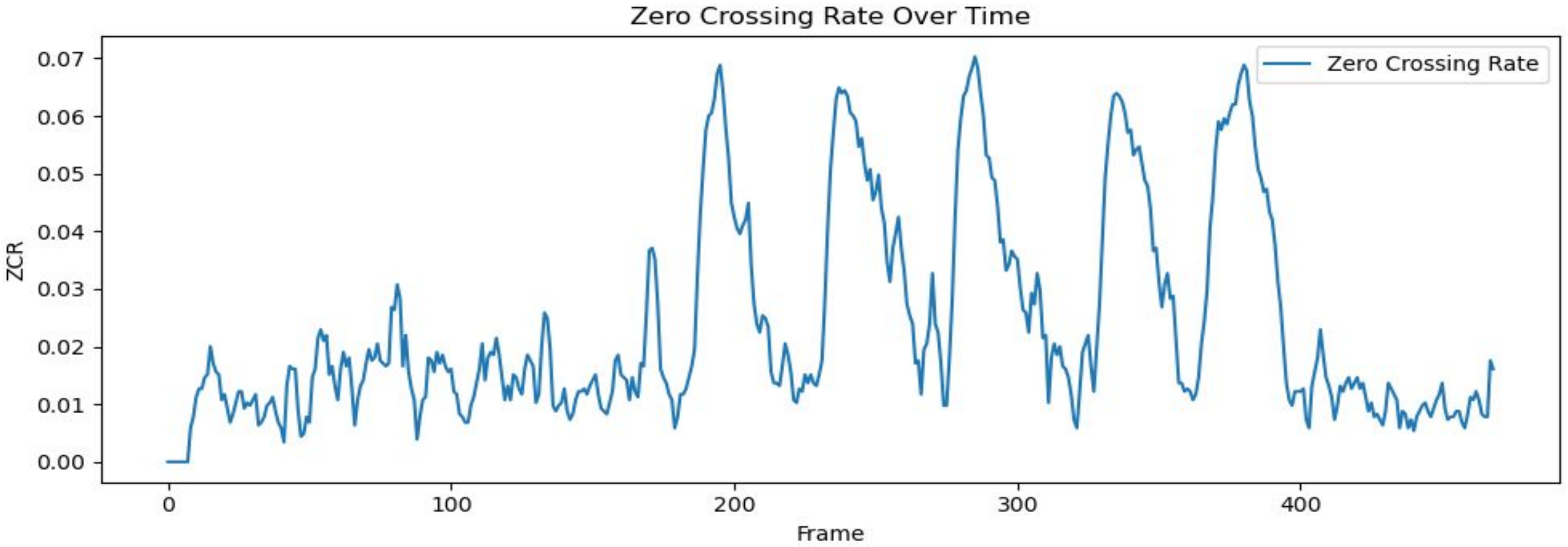
- Log Mel Spectrogram

  Human-hearing-inspired frequency representation

- MFCC (Mel-Frequency Cepstral Coefficients)

  DCT of log mel → captures timbre (spectral envelope)





- Zero Crossing Rate

  Counts how often the signal crosses zero

  Reflects roughness / percussiveness

Baseline：

- LogisticRegression accuracy: 6.5%
- RandomForest accuracy: 21%
- Guessing: 2%





2-channel(log-mel+delta),flattened

CNN from scratch..Things to Consider：

Input feature？

◎ use preprocessed dataset, with audio augmentation and slicing):

◎ how many channels to use? (e.g., log-Mel + delta-ZCR + MFCC)

Layer Design:Kernel size / Stride?

◎ Small kernels capture local features; large kernels capture broader time-frequency patterns.

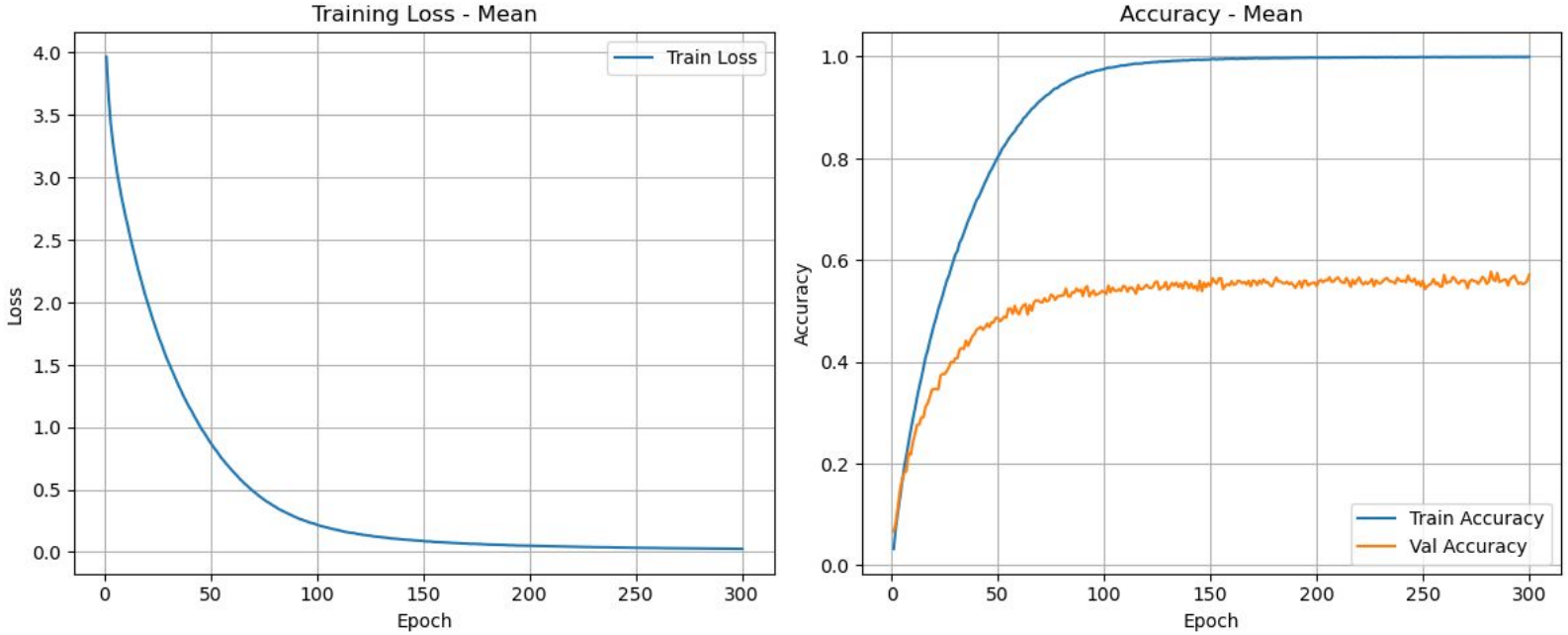
◎ considering Convolution Dimensions:time and frequency axes.

Other techniques：

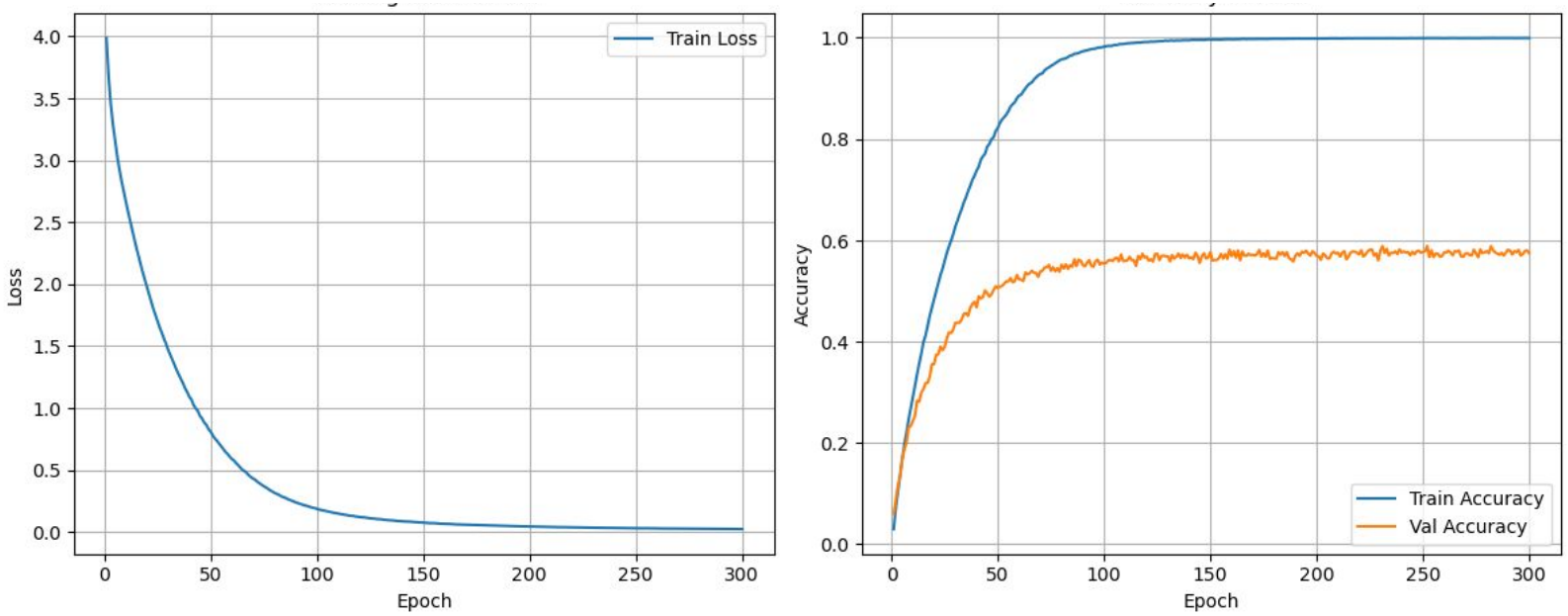
◎ Loss Function:Cross-entropy

◎ 5-fold cross-validation

◎ Regularization:(Batch Normalization / Dropout)

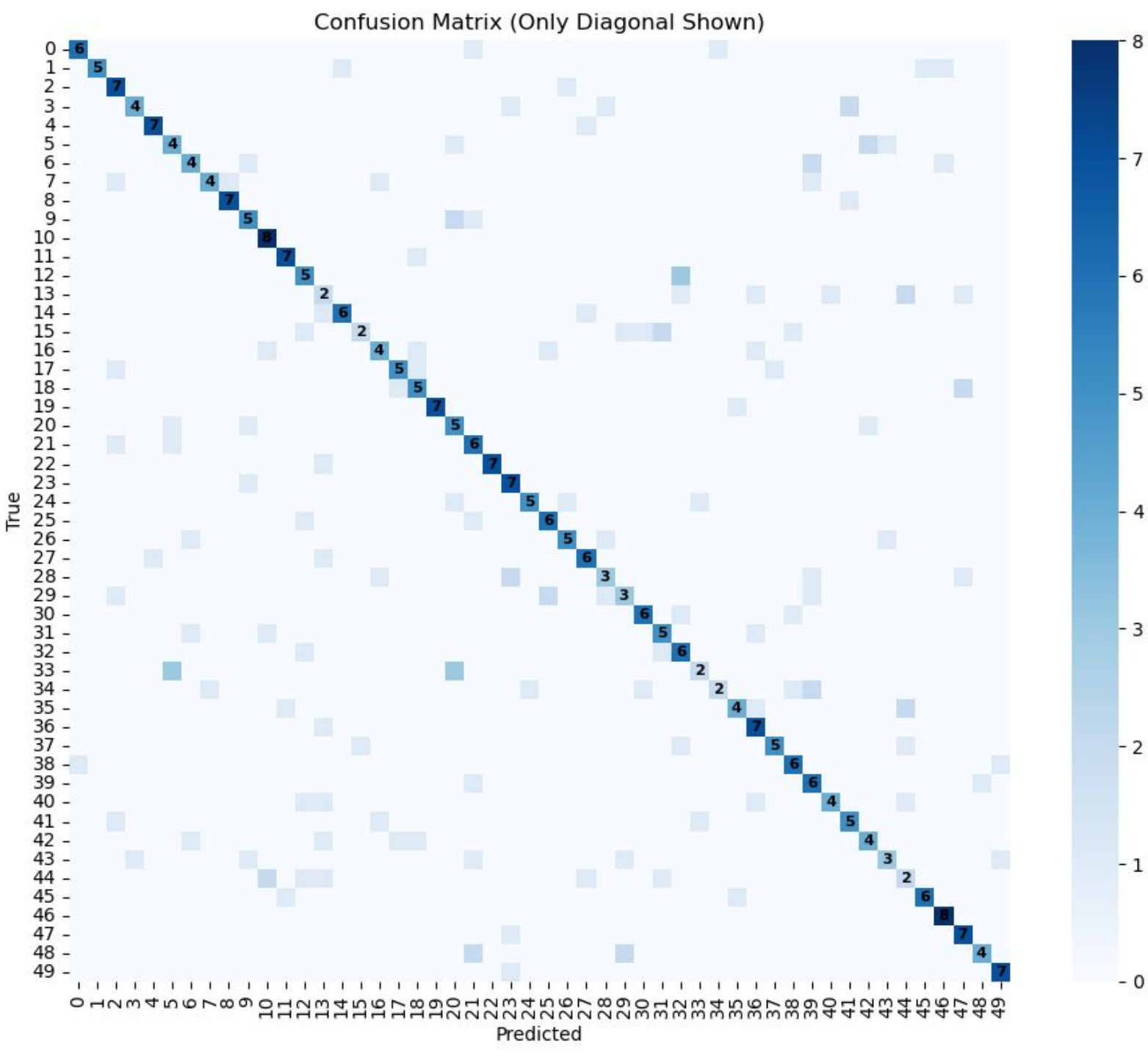


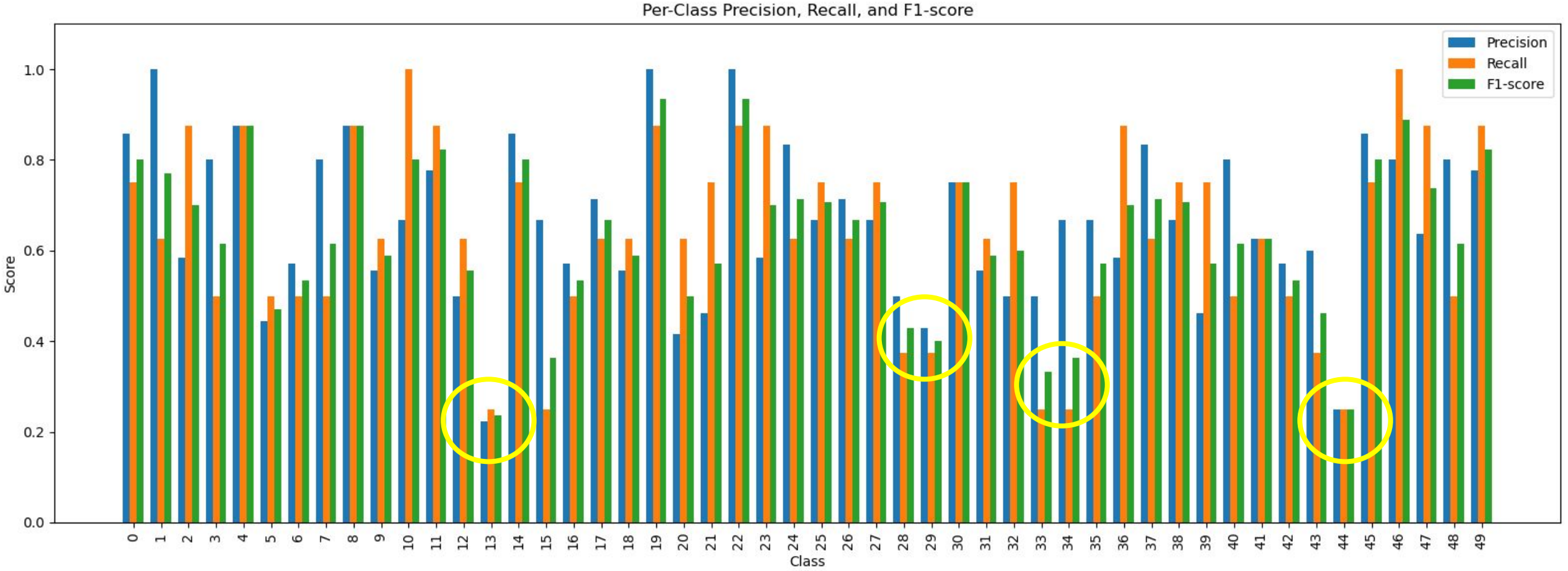
2channel features:[log-mel,delta],,mean val_acc=0.5153



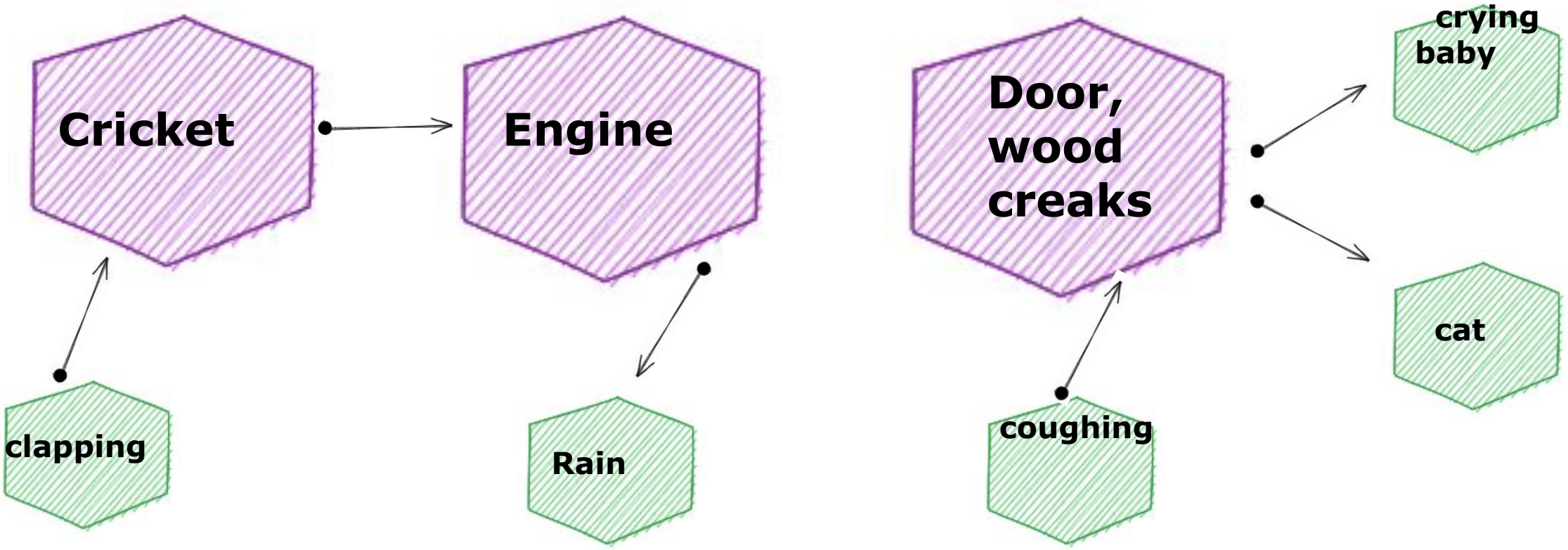
4channel features:[log-mel,delta,zcr,mfcc],mean val_acc=0.6050

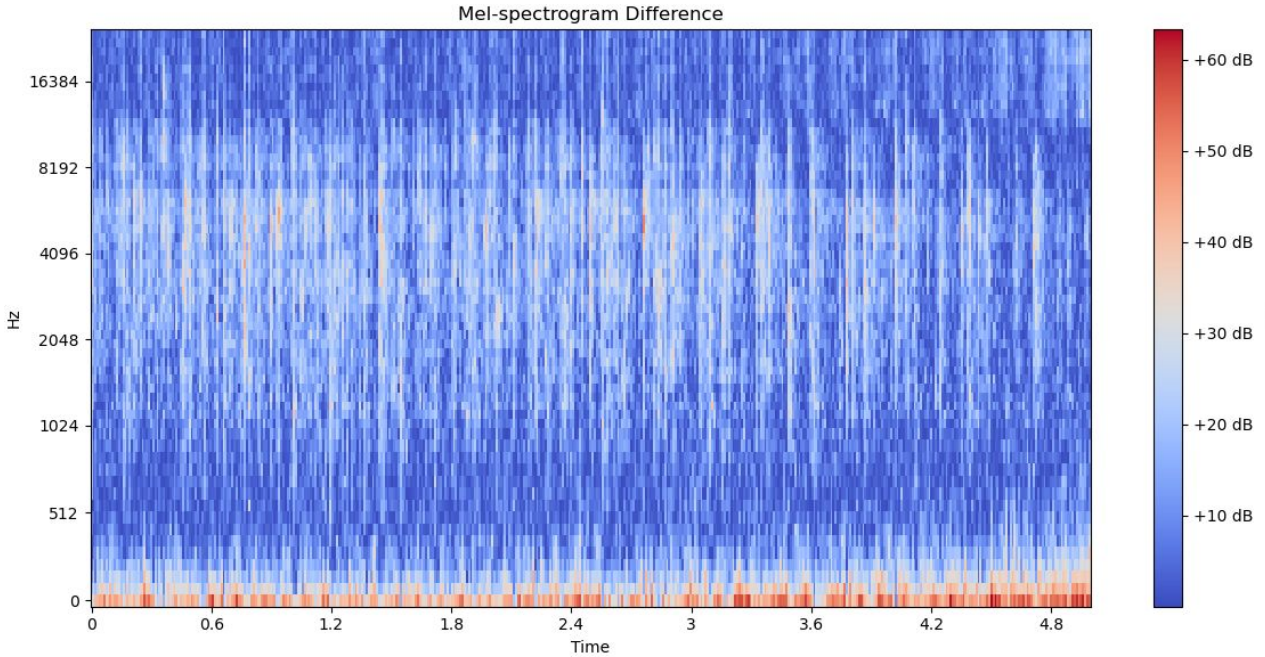
mean val_acc=60.5%
(5 folds)

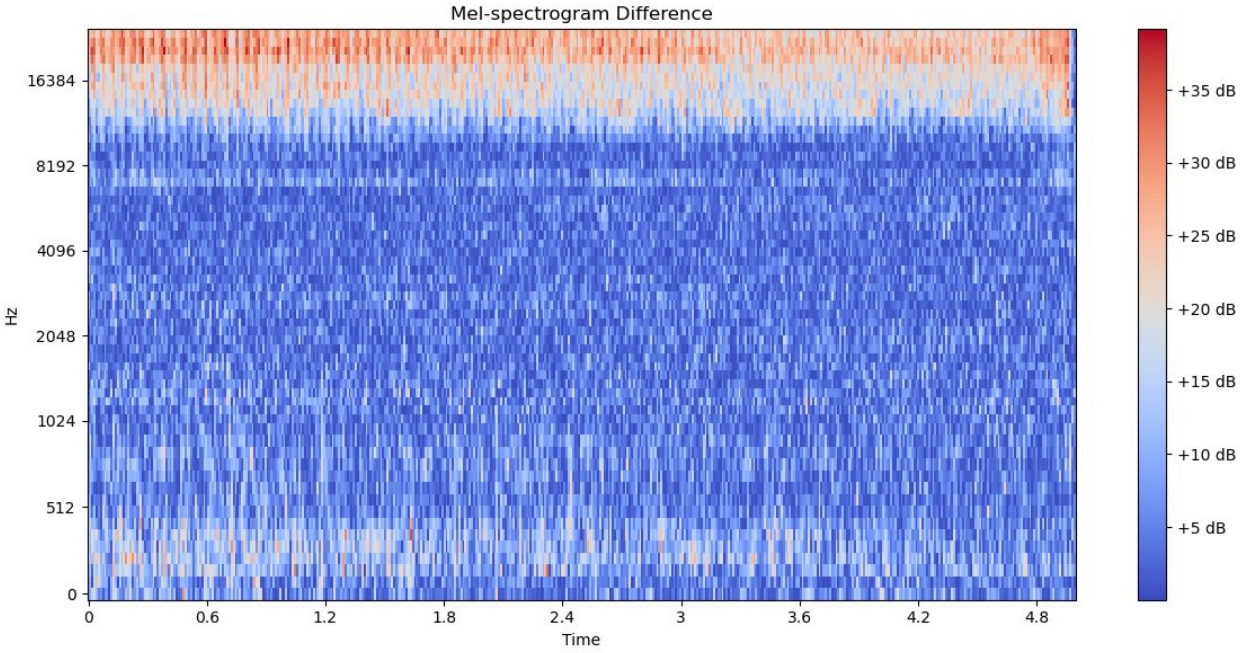

Confusion Matrix (Only Diagonal Shown)
True
Predicted

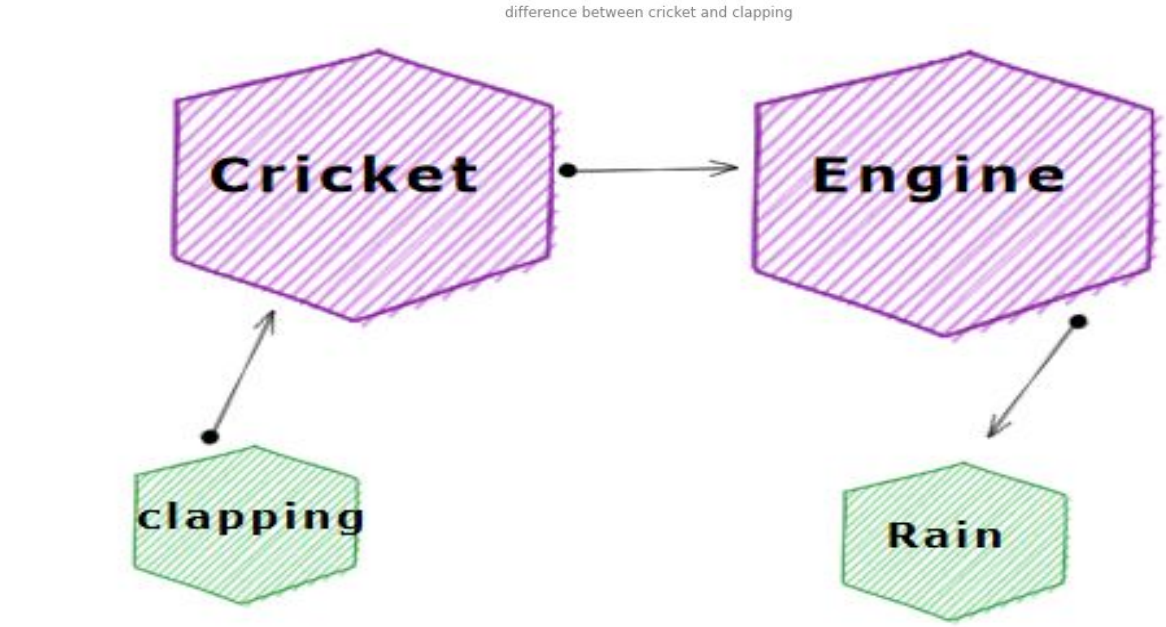

Per-Class Precision, Recall, and F1-score
Precision
Recall
F1-score
Score
Class
0.0
0.2
0.4
0.6
0.8
1.0
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49

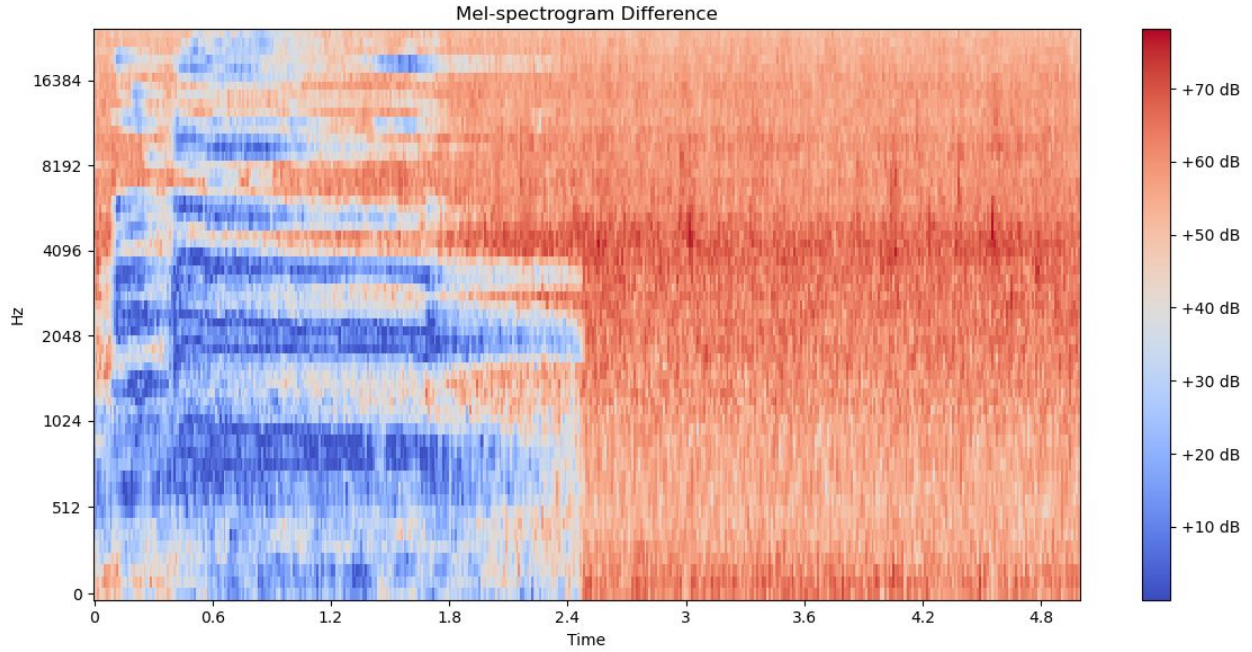
Amongst the highest F1 score:




Mel-spectrogram Difference
Hz
16384
8192
4096
2048
1024
512
0
0
0.6
1.2
1.8
2.4
3
3.6
4.2
4.8
Time
+60 dB
+50 dB
+40 dB
+30 dB
+20 dB
+10 dB


Mel-spectrogram Difference
Hz
16384
8192
4096
2048
1024
512
0
0
0.6
1.2
1.8
2.4
3
3.6
4.2
4.8
Time
+35 dB
+30 dB
+25 dB
+20 dB
+15 dB
+10 dB
+5 dB


difference between cricket and clapping
Cricket
Engine
clapping
Rain


Mel-spectrogram Difference
Hz
16384
8192
4096
2048
1024
512
0
0
0.6
1.2
1.8
2.4
3
3.6
4.2
4.8
Time
+70 dB
+60 dB
+50 dB
+40 dB
+30 dB
+20 dB
+10 dB

Pre-trained Encoder : OpenL3

OpenL3: Deep Image & Audio Embeddings

★ Open source Python library

★ Deep image & audio embeddings using a self-supervised L3-Net architecture

★ Easy to install: `pip install openl3`

★ Image: `emb = openl3.get_image_embedding(image)`

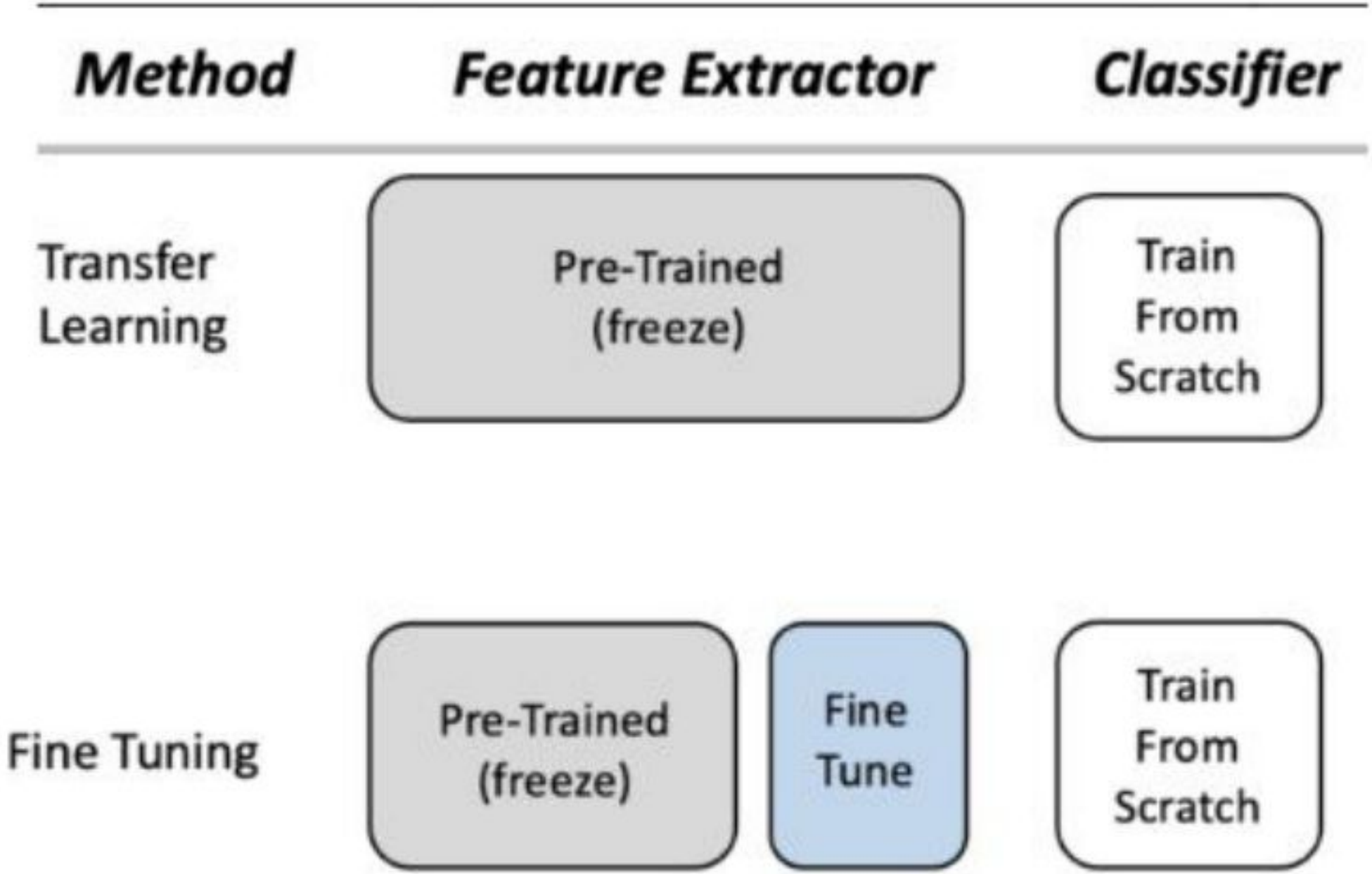
★ Audio: `emb, times = openl3.get_audio_embedding(audio, sr)`

★ Video: `openl3.process_video_file('<filepath>', output_dir='<path>')`

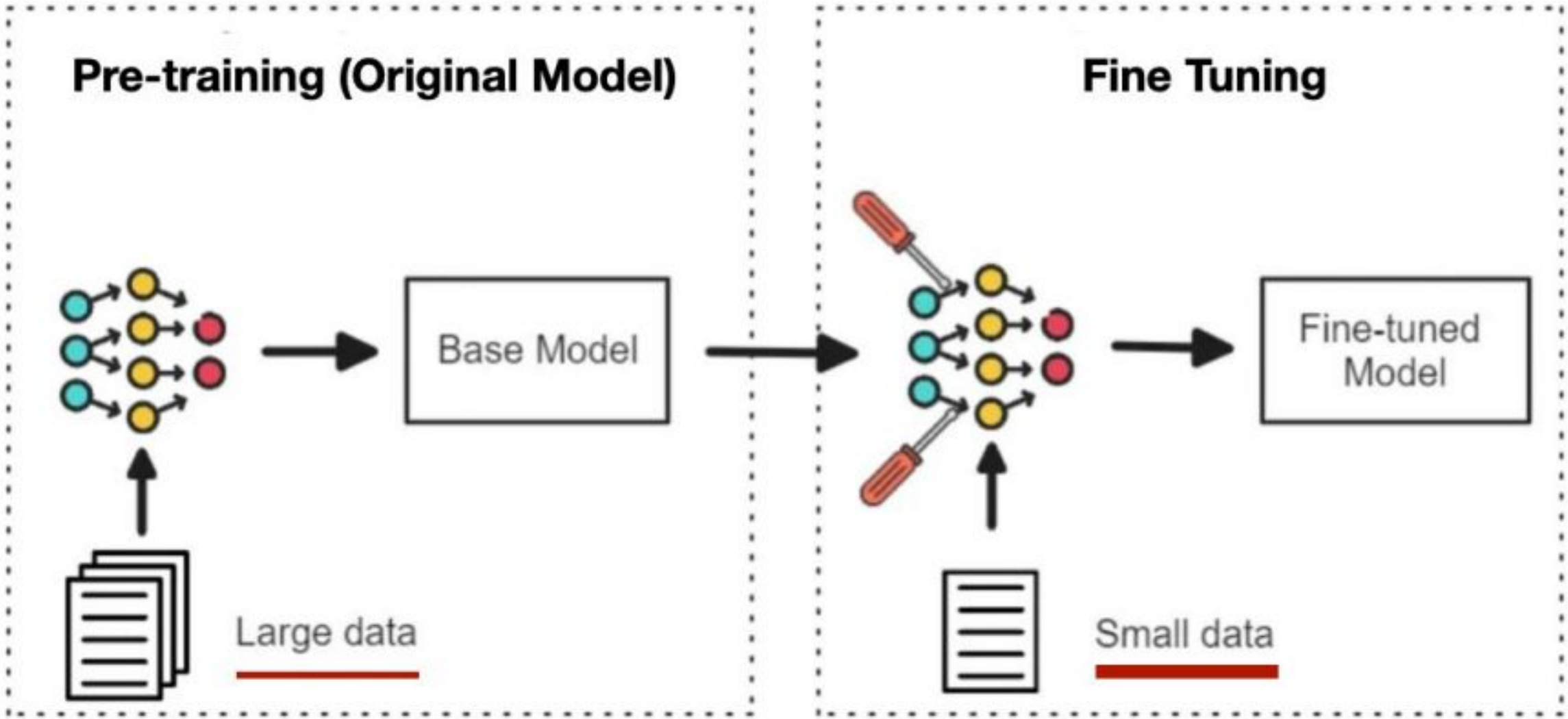
★ Learn more: https://github.com/marl/openl3 & https://openl3.readthedocs.io/

Details in paper:

"Look, Listen and Learn More: Design Choices for Deep Audio Embeddings"


Method
Feature Extractor
Classifier
Transfer Learning
Pre-Trained (freeze)
Train From Scratch
Fine Tuning
Pre-Trained (freeze)
Fine Tune
Train From Scratch

FINE-TUNING


Pre-training (Original Model)
Base Model
Large data
Fine Tuning
Fine-tuned Model
Small data

Phase C: Evaluation

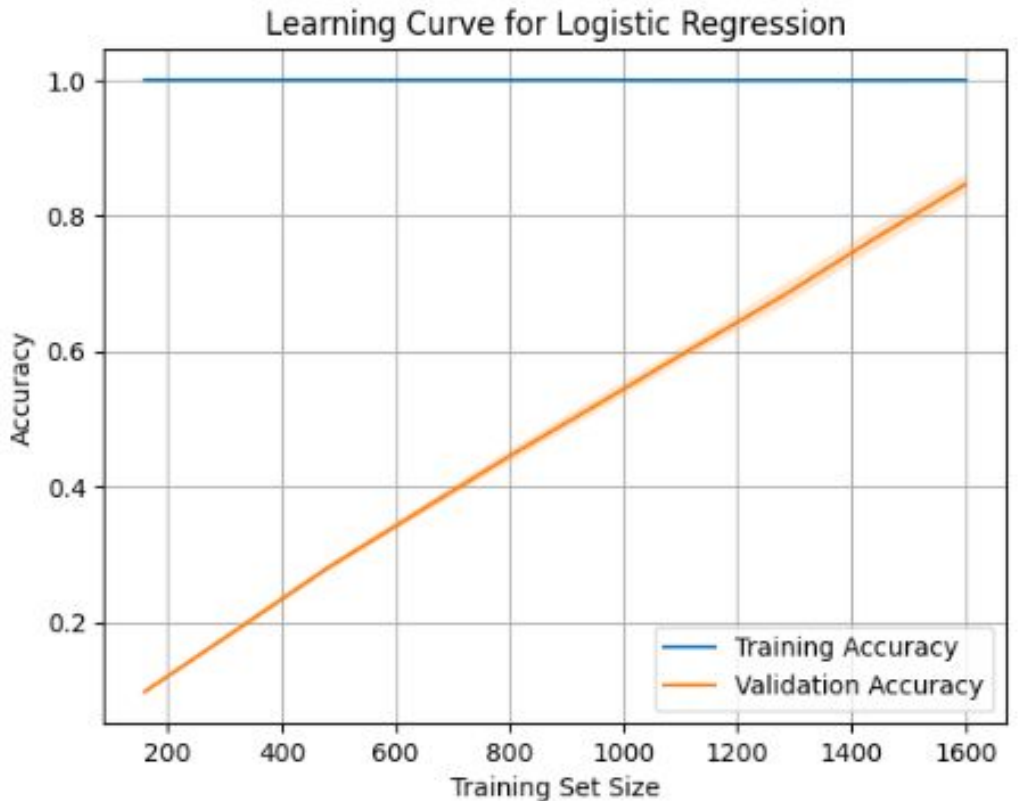
Starting Point: Scikit-learn LogisticRegression()

A simple logistic regression on the features outputted by the default openl3 model gave us a **benchmark** with 86% accuracy!

Problem:
This class gives very little feedback, and no tweakable parameters for improvement

With the following attempts, we were testing if we could surpass this model by adding more tunable layers

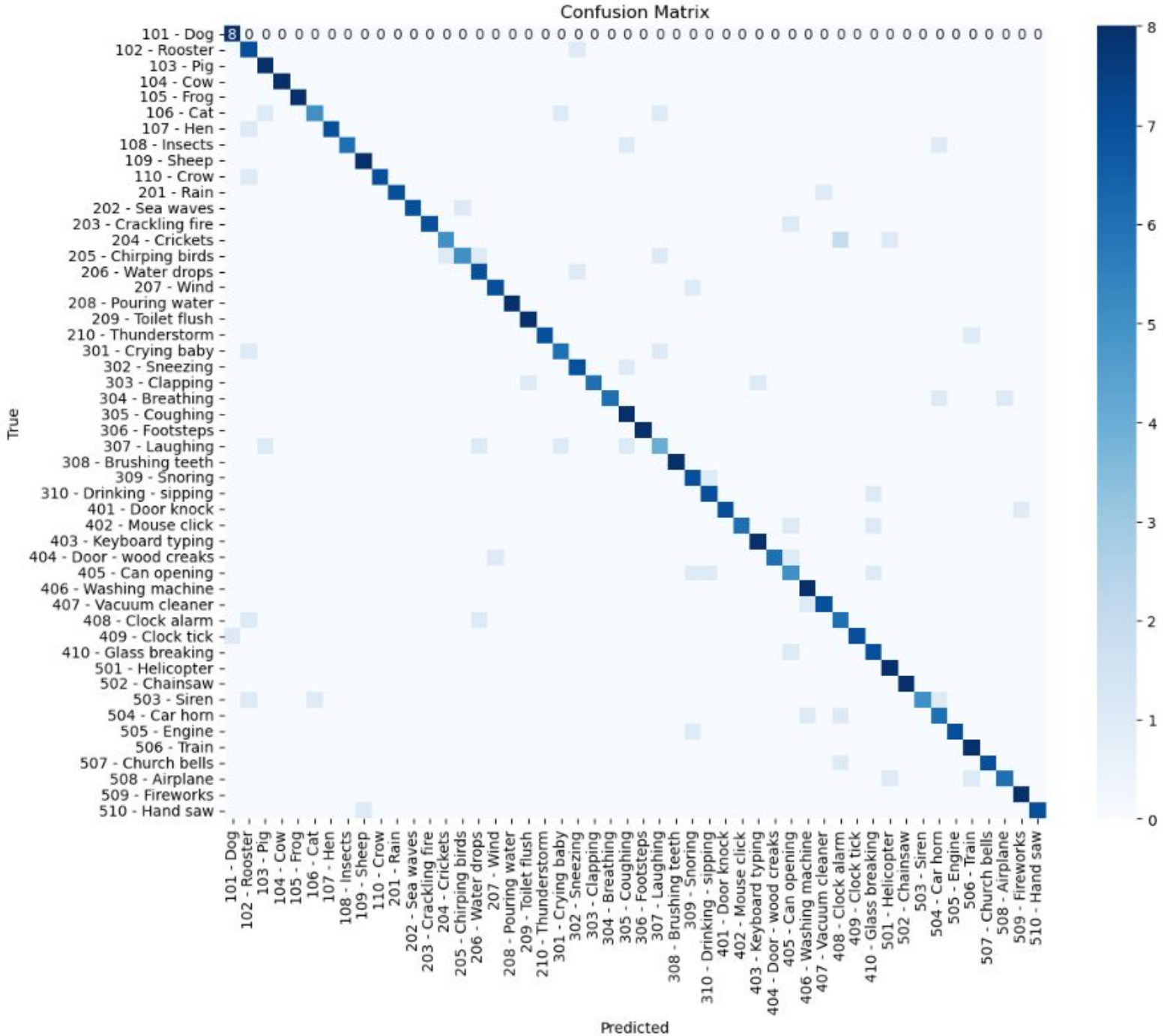
Accuracy: 0.855



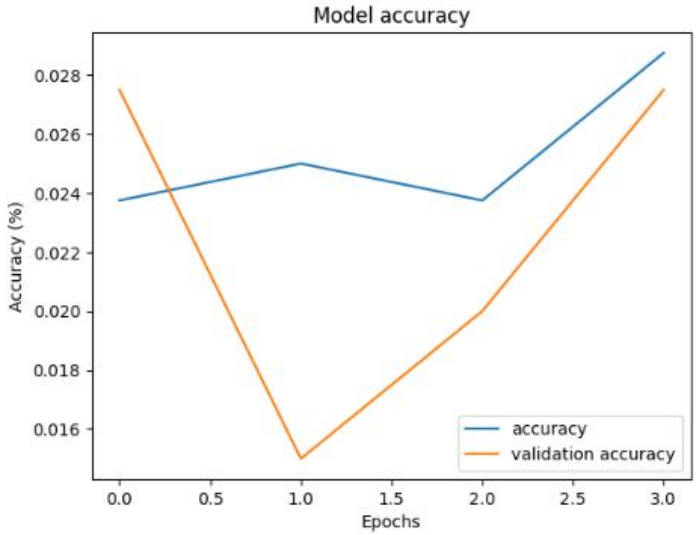
Classification Report:

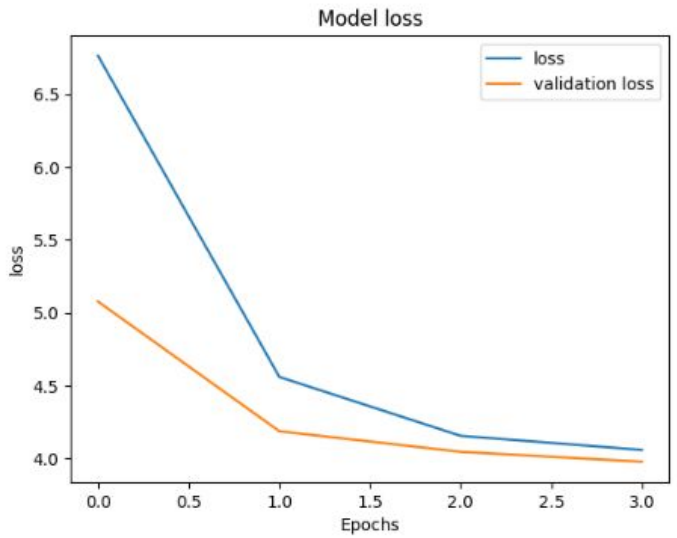
	precision	recall	f1-score	support
101 - Dog	0.89	1.00	0.94	8
102 - Rooster	0.58	0.88	0.70	8
103 - Pig	0.89	1.00	0.94	8
104 - Cow	1.00	1.00	1.00	8
105 - Frog	1.00	1.00	1.00	8
106 - Cat	0.83	0.62	0.71	8
107 - Hen	1.00	0.88	0.93	8
108 - Insects	1.00	0.75	0.86	8
109 - Sheep	0.89	1.00	0.94	8
110 - Crow	1.00	0.88	0.93	8
201 - Rain	1.00	0.88	0.93	8
202 - Sea waves	1.00	0.88	0.93	8
203 - Crackling fire	1.00	0.88	0.93	8
204 - Crickets	0.83	0.62	0.71	8
205 - Chirping birds	0.83	0.62	0.71	8
206 - Water drops	0.70	0.88	0.78	8
207 - Wind	0.88	0.88	0.88	8
208 - Pouring water	0.89	1.00	0.94	8
209 - Toilet flush	0.88	0.88	0.88	8
210 - Thunderstorm	1.00	0.88	0.93	8
301 - Crying baby	0.67	0.75	0.71	8
302 - Sneezing	0.78	0.88	0.82	8
303 - Clapping	1.00	0.75	0.86	8
304 - Breathing	1.00	0.75	0.86	8
305 - Coughing	0.73	1.00	0.84	8
306 - Footsteps	1.00	1.00	1.00	8
307 - Laughing	0.57	0.50	0.53	8
308 - Brushing teeth	1.00	1.00	1.00	8
309 - Snoring	0.78	0.88	0.82	8
310 - Drinking - sipping	0.78	0.88	0.82	8
401 - Door knock	1.00	0.88	0.93	8
402 - Mouse click	0.86	0.75	0.80	8
403 - Keyboard typing	0.88	0.88	0.88	8
404 - Door - wood creaks	1.00	0.75	0.86	8
405 - Can opening	0.56	0.62	0.59	8
406 - Washing machine	0.80	1.00	0.89	8
407 - Vacuum cleaner	0.88	0.88	0.88	8
408 - Clock alarm	0.60	0.75	0.67	8
409 - Clock tick	1.00	0.88	0.93	8
410 - Glass breaking	0.70	0.88	0.78	8
501 - Helicopter	0.80	1.00	0.89	8
502 - Chainsaw	0.89	1.00	0.94	8
503 - Siren	1.00	0.62	0.77	8
504 - Car horn	0.67	0.75	0.71	8
505 - Engine	1.00	0.88	0.93	8
506 - Train	0.80	1.00	0.89	8
507 - Church bells	1.00	0.88	0.93	8
508 - Airplane	0.86	0.75	0.80	8
509 - Fireworks	0.89	1.00	0.94	8
510 - Hand saw	1.00	0.88	0.93	8
accuracy			0.85	400
macro avg	0.87	0.85	0.86	400
weighted avg	0.87	0.85	0.86	400

Logistic Regression



Take 1: Attempting to recreate the results of the Scikit-learn Logistic Regression





Feeble results
Much slo

```
base_model = openl3.models.load_audio_embedding_model(
    input_repr=input_repr,
    content_type=content_type,
    embedding_size=embedding_size,
    frontend='kapre'
)

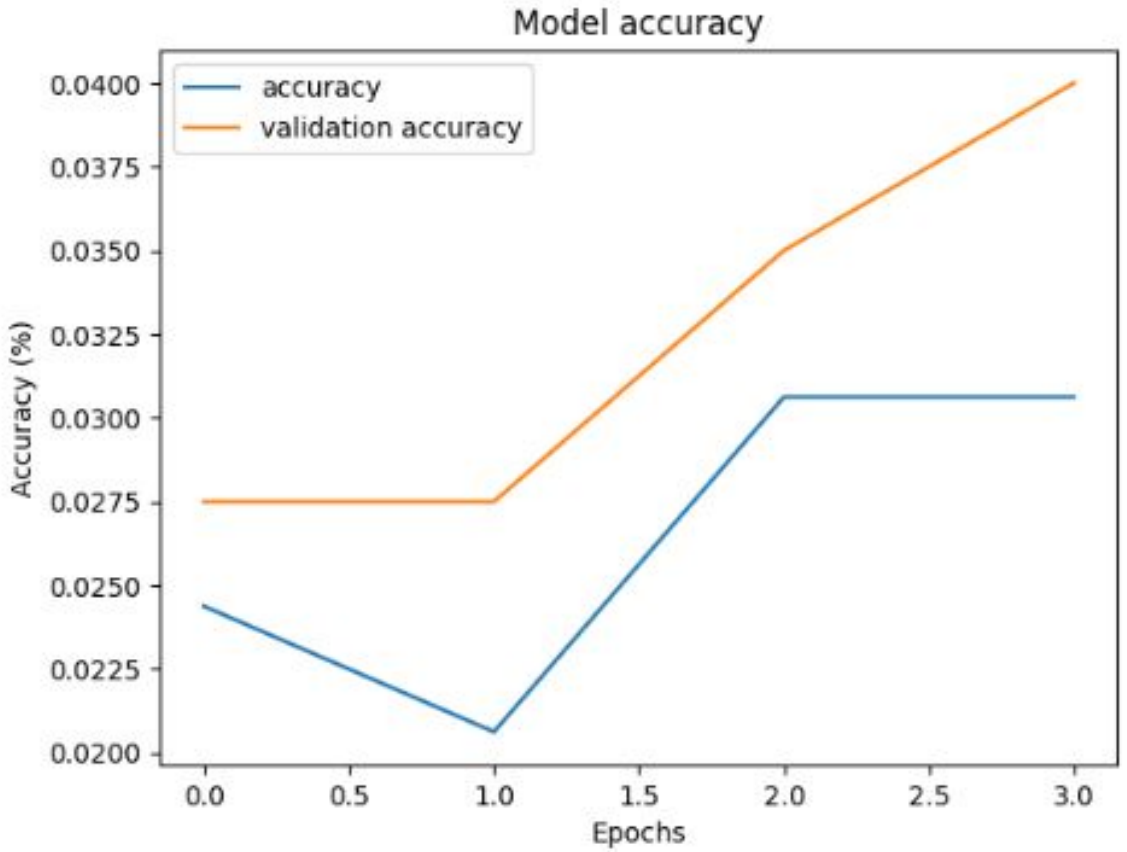

output = tf.keras.layers.Dense(num_classes, activation='softmax')(base_model.output)
model = tf.keras.Model(inputs=base_model.input, outputs=output)

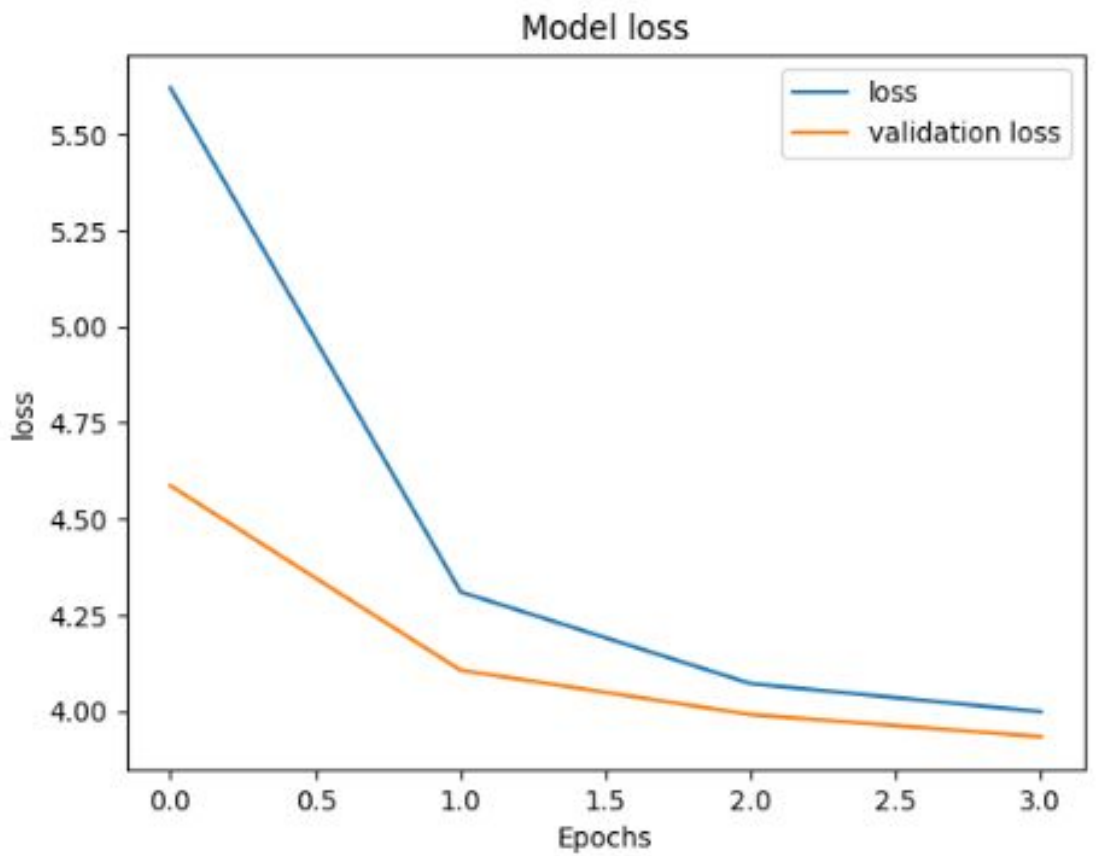
base_model.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Take 2: using the same model, but changing Y into a one-hot representation of the labels

```python
def tf_dataset(X, Y, batch_size=50):
    dataset = tf.data.Dataset.from_tensor_slices((X, Y))
    dataset = dataset.map(wrapped_preprocess, num_parallel_calls=tf.data.AUTOTUNE)
    dataset = dataset.map(lambda x, y: (x, tf.one_hot(y, depth=num_classes)))
    return dataset.batch(batch_size).prefetch(tf.data.AUTOTUNE)
```



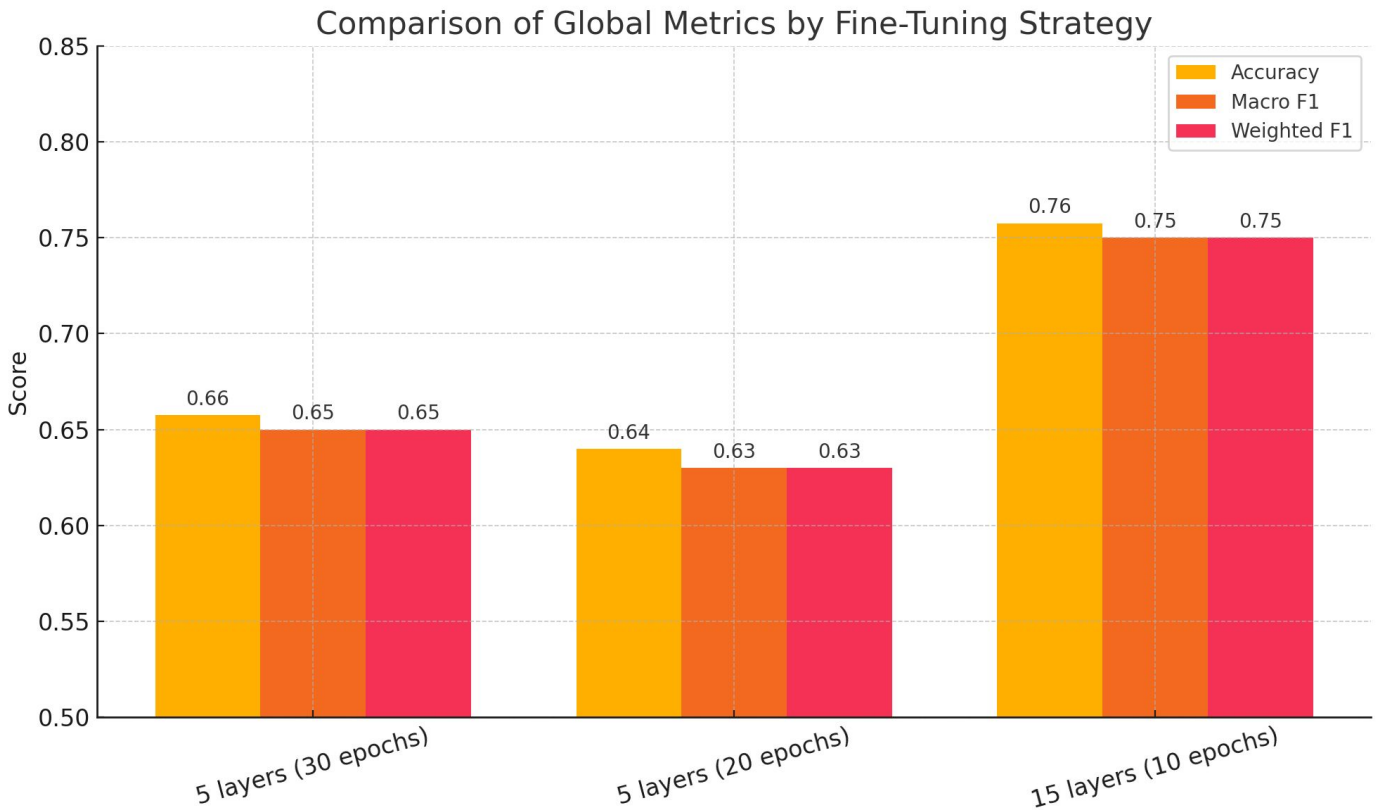


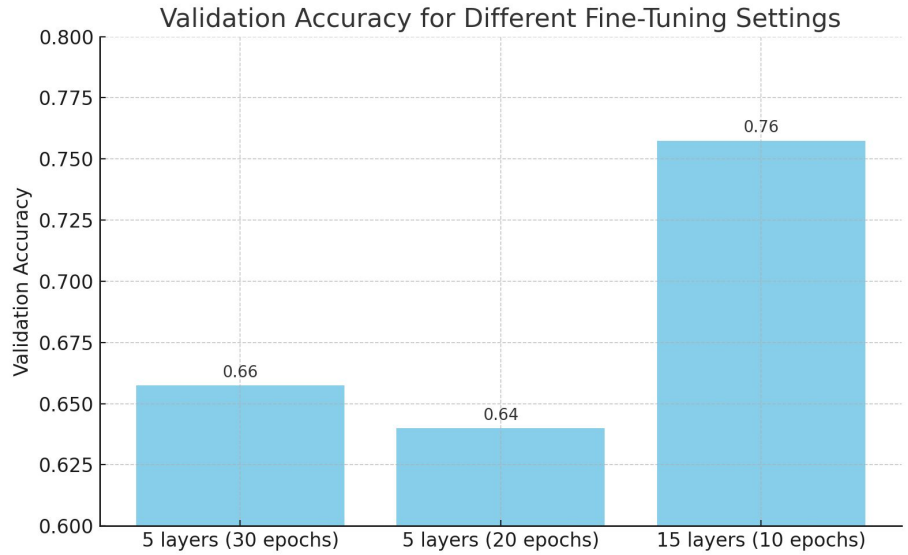
Take 3:
We progressed by adding a from-scratch model to tweak the feature space of the base (openl3) model

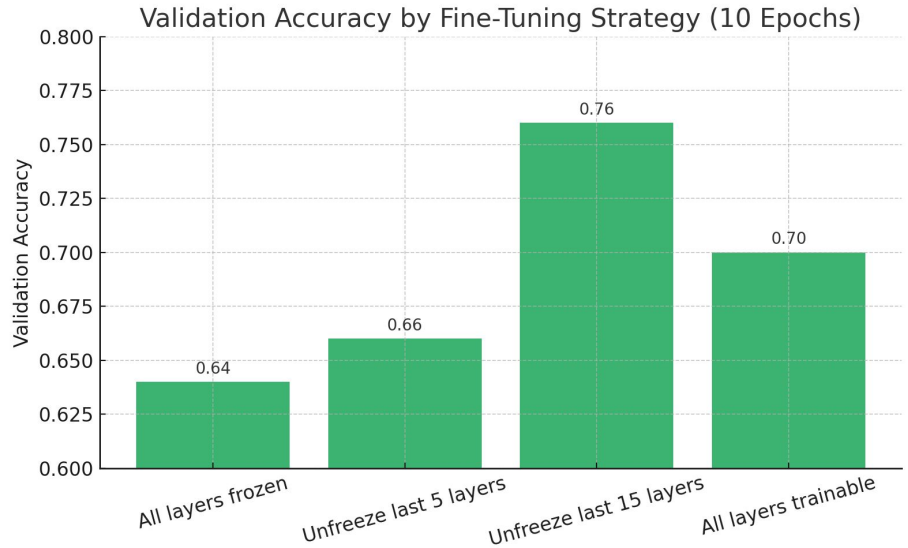
```
--- Running experiment: C1.B (0 layers) ---
Epoch 1/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 903s 4s/step - accuracy: 0.0399 - loss: 4.2552 - val_accuracy: 0.1250 - val_loss: 3.2818
Epoch 2/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 925s 5s/step - accuracy: 0.1163 - loss: 3.2786 - val_accuracy: 0.1575 - val_loss: 2.9829
Epoch 3/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 938s 5s/step - accuracy: 0.2886 - loss: 2.5104 - val_accuracy: 0.2925 - val_loss: 2.6554
Epoch 4/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 1138s 6s/step - accuracy: 0.4746 - loss: 1.8534 - val_accuracy: 0.4625 - val_loss: 1.9980
Epoch 5/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 1452s 7s/step - accuracy: 0.6031 - loss: 1.3900 - val_accuracy: 0.4250 - val_loss: 2.1882
Epoch 6/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 1244s 6s/step - accuracy: 0.7008 - loss: 1.0381 - val_accuracy: 0.4950 - val_loss: 1.8891
Epoch 7/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 1786s 9s/step - accuracy: 0.7788 - loss: 0.7156 - val_accuracy: 0.4850 - val_loss: 2.1588
Epoch 8/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 1413s 7s/step - accuracy: 0.8165 - loss: 0.5945 - val_accuracy: 0.5050 - val_loss: 2.0593
Epoch 9/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 975s 5s/step - accuracy: 0.8802 - loss: 0.3946 - val_accuracy: 0.6575 - val_loss: 1.5821
Epoch 10/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 1008s 5s/step - accuracy: 0.8954 - loss: 0.3340 - val_accuracy: 0.5850 - val_loss: 1.6897
```

```
--- Running experiment: C2a (10 layers) ---
2025-06-01 20:06:57.753950: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status:
Epoch 1/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 351s 2s/step - accuracy: 0.0409 - loss: 4.4011 - val_accuracy: 0.2600 - val_loss: 2.7310
Epoch 2/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 350s 2s/step - accuracy: 0.3387 - loss: 2.5295 - val_accuracy: 0.5750 - val_loss: 1.4689
Epoch 3/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 345s 2s/step - accuracy: 0.6036 - loss: 1.3494 - val_accuracy: 0.6875 - val_loss: 1.0798
Epoch 4/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 351s 2s/step - accuracy: 0.7903 - loss: 0.7067 - val_accuracy: 0.6975 - val_loss: 0.9851
Epoch 5/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 393s 2s/step - accuracy: 0.9026 - loss: 0.3548 - val_accuracy: 0.7400 - val_loss: 0.8847
Epoch 6/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 393s 2s/step - accuracy: 0.9506 - loss: 0.1796 - val_accuracy: 0.7725 - val_loss: 0.7822
Epoch 7/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 379s 2s/step - accuracy: 0.9690 - loss: 0.1162 - val_accuracy: 0.7550 - val_loss: 0.8836
Epoch 8/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 416s 2s/step - accuracy: 0.9737 - loss: 0.1188 - val_accuracy: 0.7725 - val_loss: 0.8184
Epoch 9/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 356s 2s/step - accuracy: 0.9924 - loss: 0.0453 - val_accuracy: 0.7525 - val_loss: 1.0190
Epoch 10/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 367s 2s/step - accuracy: 0.9836 - loss: 0.0648 - val_accuracy: 0.7800 - val_loss: 0.8354
```

```
--- Running experiment: C2b (15 layers) ---
2025-06-01 21:09:30.907470: I tensorflow/core/framework/local_rendezvous.cc:407] Local rendezvous is aborting with status: O
Epoch 1/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 456s 2s/step - accuracy: 0.0406 - loss: 4.2581 - val_accuracy: 0.2375 - val_loss: 2.6668
Epoch 2/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 466s 2s/step - accuracy: 0.2941 - loss: 2.5914 - val_accuracy: 0.5125 - val_loss: 1.6791
Epoch 3/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 487s 2s/step - accuracy: 0.5561 - loss: 1.5497 - val_accuracy: 0.6300 - val_loss: 1.2391
Epoch 4/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 553s 3s/step - accuracy: 0.7187 - loss: 0.9286 - val_accuracy: 0.6800 - val_loss: 1.0865
Epoch 5/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 469s 2s/step - accuracy: 0.8564 - loss: 0.4724 - val_accuracy: 0.6375 - val_loss: 1.3783
Epoch 6/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 471s 2s/step - accuracy: 0.9001 - loss: 0.2993 - val_accuracy: 0.6550 - val_loss: 1.4422
Epoch 7/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 458s 2s/step - accuracy: 0.9258 - loss: 0.2642 - val_accuracy: 0.7125 - val_loss: 1.1870
Epoch 8/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 495s 2s/step - accuracy: 0.9438 - loss: 0.1867 - val_accuracy: 0.7400 - val_loss: 1.0898
Epoch 9/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 582s 3s/step - accuracy: 0.9545 - loss: 0.1608 - val_accuracy: 0.6925 - val_loss: 1.3217
Epoch 10/10
200/200 ━━━━━━━━━━━━━━━━━━━━ 583s 3s/step - accuracy: 0.9515 - loss: 0.1453 - val_accuracy: 0.6775 - val_loss: 1.5328
```


Comparison of Global Metrics by Fine-Tuning Strategy
Score
Accuracy
Macro F1
Weighted F1
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.50
0.66
0.65
0.65
0.64
0.63
0.63
0.76
0.75
0.75
5 layers (30 epochs)
5 layers (20 epochs)
15 layers (10 epochs)


Validation Accuracy for Different Fine-Tuning Settings
Validation Accuracy
0.800
0.775
0.750
0.725
0.700
0.675
0.650
0.625
0.600
0.66
0.64
0.76
5 layers (30 epochs)
5 layers (20 epochs)
15 layers (10 epochs)


Validation Accuracy by Fine-Tuning Strategy (10 Epochs)
Validation Accuracy
0.800
0.775
0.750
0.725
0.700
0.675
0.650
0.625
0.600
0.64
0.66
0.76
0.70
All layers frozen
Unfreeze last 5 layers
Unfreeze last 15 layers
All layers trainable

```
Best model: C2a (10 layers)
```

